

Curso desarrollo de aplicaciones con tecnologías web

UF1842. Desarrollo y reutilización de componentes software y multimedia mediante lenguajes de guión.

Lenguajes de guión de uso general

Funciones

Manipulación de texto

Listas (arrays)

Objetos

Funciones

¿Qué son las funciones?

Las funciones son bloques de código reutilizables diseñados para realizar una tarea específica. Se definen una vez y se pueden ejecutar tantas veces como sea necesario.

Parámetros

Los **parámetros** son variables que se pasan a las funciones para que trabajen con diferentes datos. En este ejemplo el parámetro es nombre. Cuando llamas a la función, pasas un **argumento** a esa función. En este caso, "Ana" y "Carlos" son los argumentos que se pasan cuando se llaman a la función saludar.

JavaScript:

```
// Función con parámetros
function saludar(nombre) {
    console.log(`Hola, ${nombre}!`);
}

// Llamando a la función con diferentes argumentos
saludar("Ana"); // Hola, Ana!
saludar("Carlos"); // Hola, Carlos!
```

Variables locales y globales

Los operadores realizan operaciones con valores:

- **Locales:** Declaradas dentro de una función y solo accesibles dentro de esa función.
- **Globales:** Declaradas fuera de cualquier función y accesibles desde cualquier lugar.

JavaScript:

```
let globalVar = "Soy global"; // Variable global

function mostrarVar() {
  let localVar = "Soy local"; // Variable local
  console.log(globalVar); // Accede a la variable global
  console.log(localVar); // Accede a la variable local
}

mostrarVar();
console.log(globalVar); // Funciona
// console.log(localVar); // Error: localVar no está definida
```

Bibliotecas de funciones

Las bibliotecas de funciones son colecciones de funciones predefinidas o personalizadas que se agrupan en archivos externos y se pueden importar. Las hemos visto en Angular al importar funcionalidades.

Ejemplo con una biblioteca simple:

```
// archivo matematicas.js
export function sumar(a, b) {
  return a + b;
}

export function restar(a, b) {
  return a - b;
}

// archivo principal.js
import { sumar, restar } from './matematicas.js';

console.log(sumar(5, 3)); // 8
console.log(restar(9, 4)); // 5
```

Manipulación de texto

Funciones básicas para la manipulación de texto

JavaScript ofrece métodos útiles para trabajar con cadenas de texto.

javascript:

```
let texto = "Hola, mundo!";

// Convertir a mayúsculas
console.log(texto.toUpperCase()); // "HOLA, MUNDO!"

// Convertir a minúsculas
console.log(texto.toLowerCase()); // "hola, mundo!"

// Obtener longitud de la cadena
console.log(texto.length); // 12

// Reemplazar texto
console.log(texto.replace("mundo", "JavaScript")); // "Hola, JavaScript!"
```

JavaScript proporciona muchos más métodos útiles para la manipulación de texto. A continuación proporciono una tabla con algunos de los más utilizados:

Método	Descripción	Ejemplo
toUpperCase()	Convierte una cadena a mayúsculas.	<pre>let texto = "Hola"; console.log(texto.toUpperCase()); // "HOLA"</pre>
toLowerCase()	Convierte una cadena a minúsculas.	<pre>let texto = "Hola"; console.log(texto.toLowerCase()); // "hola"</pre>
length	Obtiene la longitud de la cadena.	<pre>let texto = "Hola"; console.log(texto.length); // 4</pre>
replace(search, replace)	Reemplaza una subcadena con otra.	<pre>let texto = "Hola, mundo!"; console.log(texto.replace("mundo", "JavaScript")); // "Hola, JavaScript!"</pre>
indexOf(search)	Devuelve el índice de la primera aparición de la subcadena.	<pre>let texto = "Hola, mundo!"; console.log(texto.indexOf("mundo")); // 6</pre>
lastIndexOf(search)	Devuelve el índice de la última aparición de la subcadena.	<pre>let texto = "Hola, mundo, mundo!"; console.log(texto.lastIndexOf("mundo")); // 13</pre>
charAt(index)	Devuelve el carácter en una posición específica.	<pre>let texto = "Hola"; console.log(texto.charAt(1)); // "o"</pre>
slice(start, end)	Extrae una sección de la cadena y la devuelve como nueva.	<pre>let texto = "Hola, mundo!"; console.log(texto.slice(0, 4)); // "Hola"</pre>
substring(start, end)	Devuelve una subcadena entre dos índices.	<pre>let texto = "Hola, mundo!"; console.log(texto.substring(0, 4)); // "Hola"</pre>

Método	Descripción	Ejemplo
split(separator)	Divide una cadena en un array, usando un separador.	<pre>let texto = "Hola, mundo!"; console.log(texto.split(", ")); // ["Hola", "mundo!"]</pre>
trim()	Elimina los espacios en blanco al principio y al final.	<pre>let texto = " Hola, mundo! "; console.log(texto.trim()); // "Hola, mundo!"</pre>
includes(substring)	Verifica si una cadena contiene otra.	<pre>let texto = "Hola, mundo!"; console.log(texto.includes("mundo")); // true</pre>
startsWith(substring)	Verifica si una cadena comienza con la subcadena especificada.	<pre>let texto = "Hola, mundo!"; console.log(texto.startsWith("Hola")); // true</pre>
endsWith(substring)	Verifica si una cadena termina con la subcadena especificada.	<pre>let texto = "Hola, mundo!"; console.log(texto.endsWith("mundo!")); // true</pre>

Introducción y validación de texto

Se puede pedir al usuario que introduzca texto y luego validarlo.

javascript:

```
let entrada = prompt("Introduce tu nombre:");

// Validar que no esté vacío
if (entrada.trim() === "") {
    alert("Por favor, introduce un nombre válido.");
} else {
    alert(`Hola, ${entrada}!`);
}
```

Tabla con los métodos de validación comunes y sus descripciones:

Esta tabla resume los métodos más comunes para validar entradas en JavaScript. Cada uno tiene un propósito específico dependiendo de la necesidad de validación.

Método	Descripción	Ejemplo
trim()	Elimina los espacios en blanco al principio y al final de una cadena.	<pre>let nombre = " Ana "; nombre = nombre.trim(); console.log(nombre); // "Ana"</pre>
isNaN()	Verifica si el valor no es un número (NaN).	<pre>let edad = prompt("Introduce tu edad:"); if (isNaN(edad)) { alert("Por favor, introduce un número válido."); }</pre>

Método	Descripción	Ejemplo
length	Devuelve la longitud de la cadena. Se usa para validar la cantidad de caracteres de la entrada.	<pre> let palabra = prompt("Introduce una palabra:"); if (palabra.length < 3) { alert("Debe tener al menos 3 caracteres."); } </pre>
match()	Compara la cadena con una expresión regular. Utilizado para validar patrones como emails, contraseñas, etc.	<pre> let email = prompt("Introduce tu email:"); let regex = /^[a-zA-Z0-9._-]+@[a-zA0-9.-]+\.[a-zA-Z]{2,6}\$/; if (!email.match(regex)) { alert("Email inválido."); } </pre>
includes()	Verifica si la cadena contiene una subcadena.	<pre> let palabra = prompt("Introduce una palabra que contenga 'Java':"); if (!palabra.includes("Java")) { alert("Debe contener 'Java'."); } </pre>

Listas (arrays)

Creación de arrays básicos

Un array es una lista de elementos, como números, cadenas o incluso otros arrays.

javascript:

```
// Array básico
let frutas = ["Manzana", "Banana", "Cereza"];
console.log(frutas[0]); // "Manzana"
console.log(frutas.length); // 3
```

Arrays multidimensionales

Un array multidimensional es un array que contiene otros arrays.

javascript:

```
let matriz = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

console.log(matriz[1][2]); // 6 (segunda fila, tercera columna)
```

Tratamiento de arrays mediante bucles

Recorrer un array para realizar acciones con sus elementos.

javascript

```
// Declaramos un array llamado 'numeros' que contiene una lista de números.
let numeros = [10, 20, 30, 40, 50];

// Iniciamos un bucle for para recorrer todos los elementos del array.
// La estructura del bucle for consta de tres partes:
// 1. Inicialización: 'let i = 0' establece la variable i en 0 (inicio del bucle).
// 2. Condición: 'i < numeros.length' indica que el bucle se ejecutará mientras i sea menor que la
longitud del array.
// En este caso, numeros.length es 5, ya que el array tiene 5 elementos.
// 3. Incremento: 'i++' aumenta el valor de i en 1 después de cada iteración.

for (let i = 0; i < numeros.length; i++) {

// Dentro del bucle, usamos console.log para mostrar un mensaje en la consola.
// `${numeros[i]}` usa plantillas literales para insertar el valor actual del array en la posición
i.
// Por ejemplo, cuando i es 0, el valor será numeros[0], que es 10.

console.log(`El número es: ${numeros[i]}`);

}
```

Objetos

Creación de objetos: métodos y estructuras de datos

Un objeto es una colección de propiedades y métodos.

JavaScript:

```
let coche = {  
  marca: "Toyota",  
  modelo: "Corolla",  
  año: 2020,  
  arrancar: function () {  
    return `El coche ${this.marca} ${this.modelo} está en marcha.`;  
  }  
};  
  
console.log(coche.marca); // "Toyota"  
console.log(coche.arrancar()); // "El coche Toyota Corolla está en marcha."
```

Bibliotecas de objetos

JavaScript incluye objetos integrados como Math para matemáticas avanzadas o Date para fechas.

Ejemplo con Math:

```
console.log(Math.max(10, 20, 30)); // 30
console.log(Math.random()); // Número aleatorio entre 0 y 1
console.log(Math.round(4.7)); // 5
```

Ejemplo con Date:

```
let fecha = new Date();
console.log(fecha.toString()); // "Tue Jan 16 2025"
console.log(fecha.getFullYear()); // 2025
```

Tabla con ejemplos de bibliotecas de objetos integrados en JavaScript

Esta tabla incluye objetos integrados que son esenciales para el desarrollo en JavaScript y que se utilizan en diversas situaciones, desde manejo de fechas hasta colecciones de datos.

Objeto Integrado	Descripción	Ejemplo
Math	Ofrece funciones matemáticas para cálculos como redondeo, trigonometría, etc.	<code>Math.round(3.14);</code> //redondea el número 3.14 a 3.
Date	Permite trabajar con fechas y horas.	<code>let fecha = new Date();</code> //obtiene la fecha y hora actuales.

Objeto Integrado	Descripción	Ejemplo
String	Proporciona métodos para manipular y trabajar con cadenas de texto.	<code>'Hola'.toLowerCase();</code> <code>//convierte "Hola" a "hola".</code>
Array	Proporciona métodos para manipular arrays.	<code>let arr = [1, 2, 3]; arr.push(4);</code> <code>//agrega el número 4 al final del array.</code>
JSON	Permite trabajar con JSON (JavaScript Object Notation) para convertir objetos a cadenas y viceversa.	<code>JSON.stringify({name: "John"});</code> <code>//convierte un objeto a JSON.</code>
RegExp	Permite trabajar con expresiones regulares.	<code>/abc/.test("abcdef");</code> <code>//devuelve true si "abc" está en "abcdef".</code>
Promise	Representa un valor que puede estar disponible ahora o en el futuro (utilizado para manejo de asincronía).	<code>let p = new Promise((resolve, reject) =></code> <code>resolve('Completado'));</code>
Set	Representa una colección de valores únicos.	<code>let mySet = new Set([1, 2, 3]);</code> <code>//crea un conjunto con los valores 1, 2 y 3.</code>
Map	Representa una colección de pares clave-valor.	<code>let myMap = new Map(); myMap.set('key', 'value');</code> <code>//crea un mapa con una clave 'key' y valor 'value'.</code>
WeakSet	Similar a Set, pero sus elementos son objetos y pueden ser recolectados por el Garbage Collector.	<code>let weakSet = new WeakSet(); weakSet.add({});</code>

Objeto Integrado	Descripción	Ejemplo
WeakMap	Similar a Map, pero las claves deben ser objetos, y los valores pueden ser recolectados por el Garbage Collector.	<code>let weakMap = new WeakMap(); weakMap.set({}, 'value');</code>